

# CFS

operating system

---

CFS

# CONTENTS

01

CFS 이전의 시대

---

02

CFS

---

03

CFS의 한계

---

01

CFS 이전의 시대

# O(n) 프로세스 스케줄러 [리눅스 2.4]

- 스케줄링 결정 시에 실행 가능한 모든 프로세스를 순차적으로 탐색하는 방식
  - N개의 프로세스가 있는 준비 큐를 확인함
  - 우선순위에 따라 타임 퀀텀을 기준으로 실행함
  - 문제점
    - N이 커지면?
    - 락 경합 → 병목 현상 발생
-

# O(1) 프로세스 스케줄러 [리눅스 2.6.0 ~ 2.6.22]

- 실행 중인 프로세스 수와 관계없이 상수 시간 내에 다음 프로세스를 결정함
  - 'Active'와 'Expired'라는 두 개의 우선순위 배열을 사용함
  - 비트맵으로 우선순위를 관리함
  - 문제점 해결
    - N이 커지면?
    - 각 CPU 마다 별도의 큐를 가지면 됨
  - 문제점
    - 사용자 반응이 중요한 작업은?
-

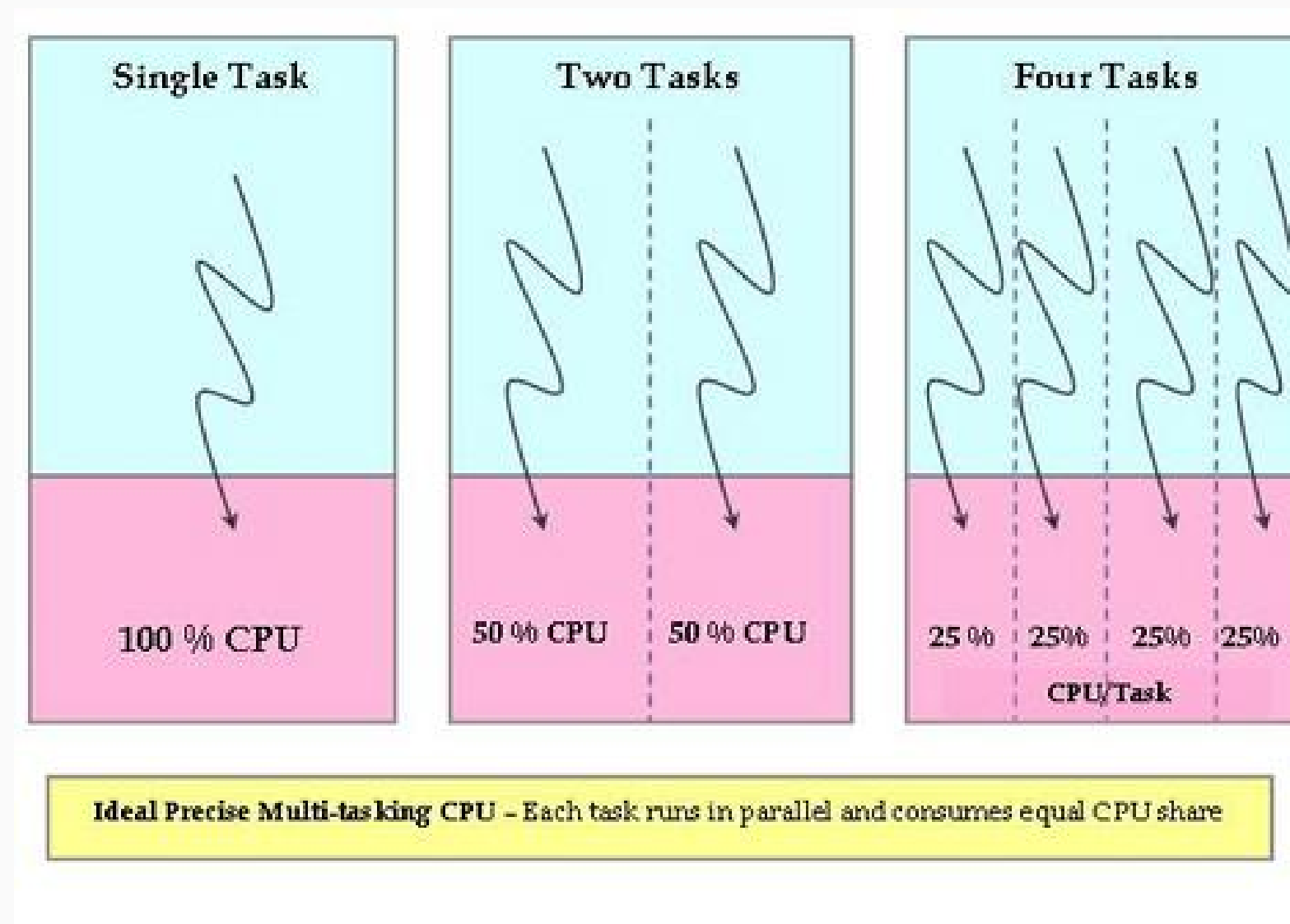




02

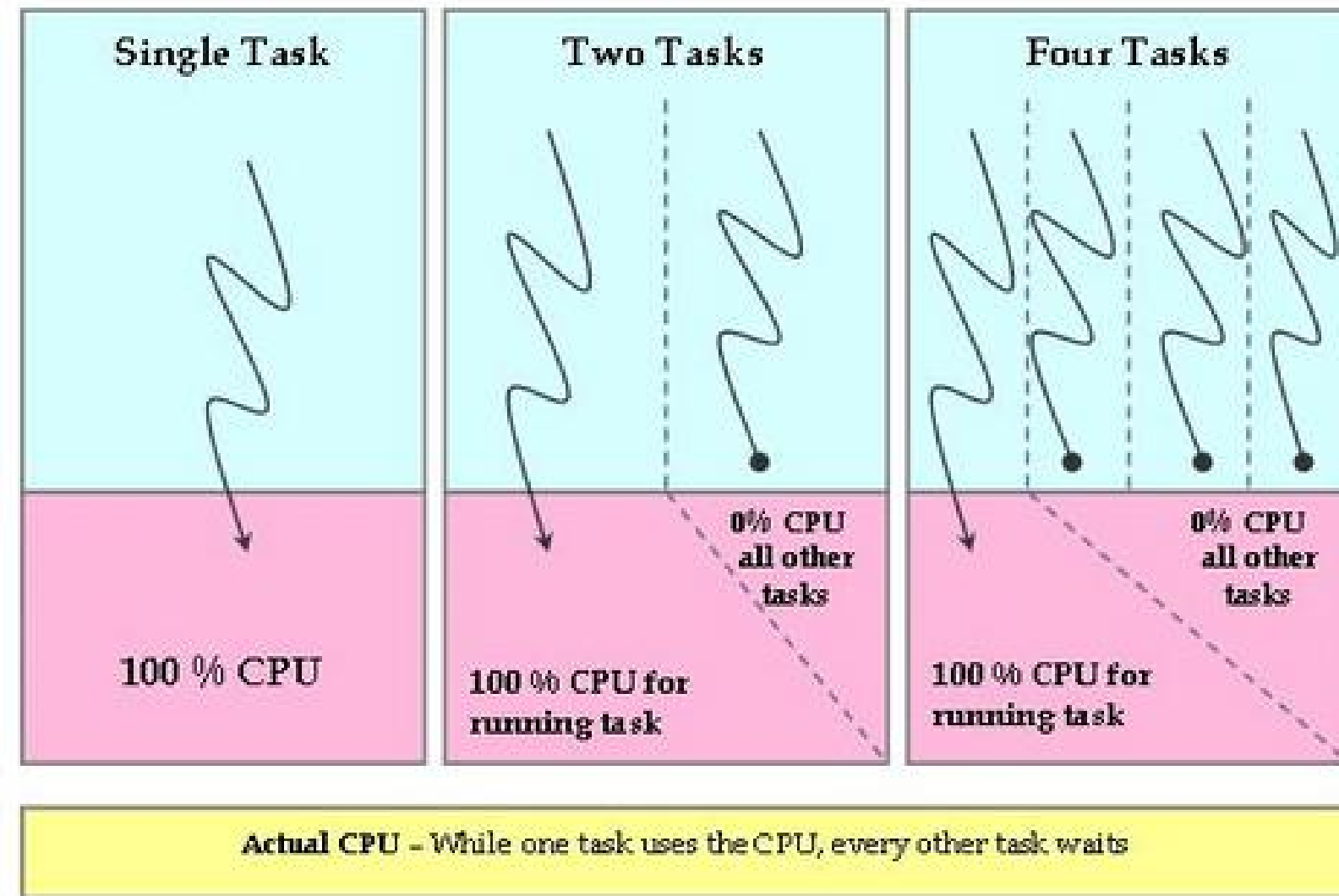
CFS

# CFS [리눅스 2.6.23 ~]



- N개의 실행 가능한 프로세스가 있다면 **이상적인** 멀티태스킹 CPU 모델은?
  - 각각의 프로세스에게 1/N의 CPU 파워를 동시에 제공해야 함

# CFS [리눅스 2.6.23 ~ ]



- 실제 CPU는 동시에 실행시킬 수가 없는데요?
  - 시간을 쪼개서 병렬 실행의 환상을 만들어 내자



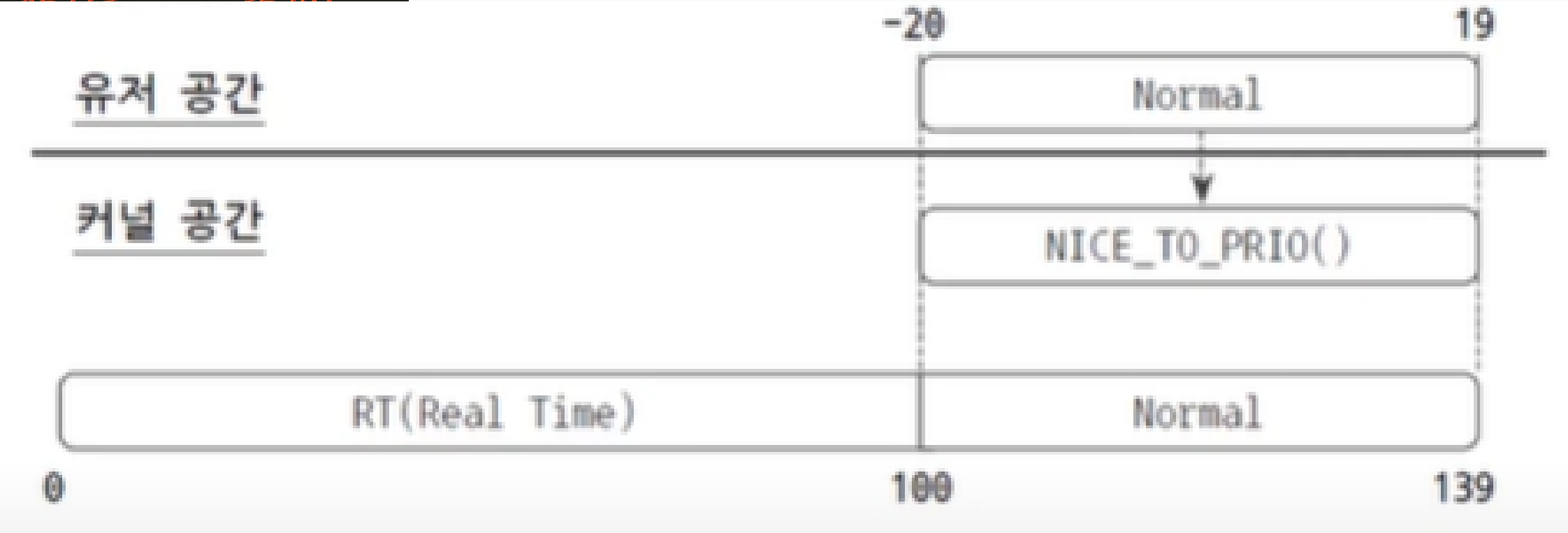
# vruntime

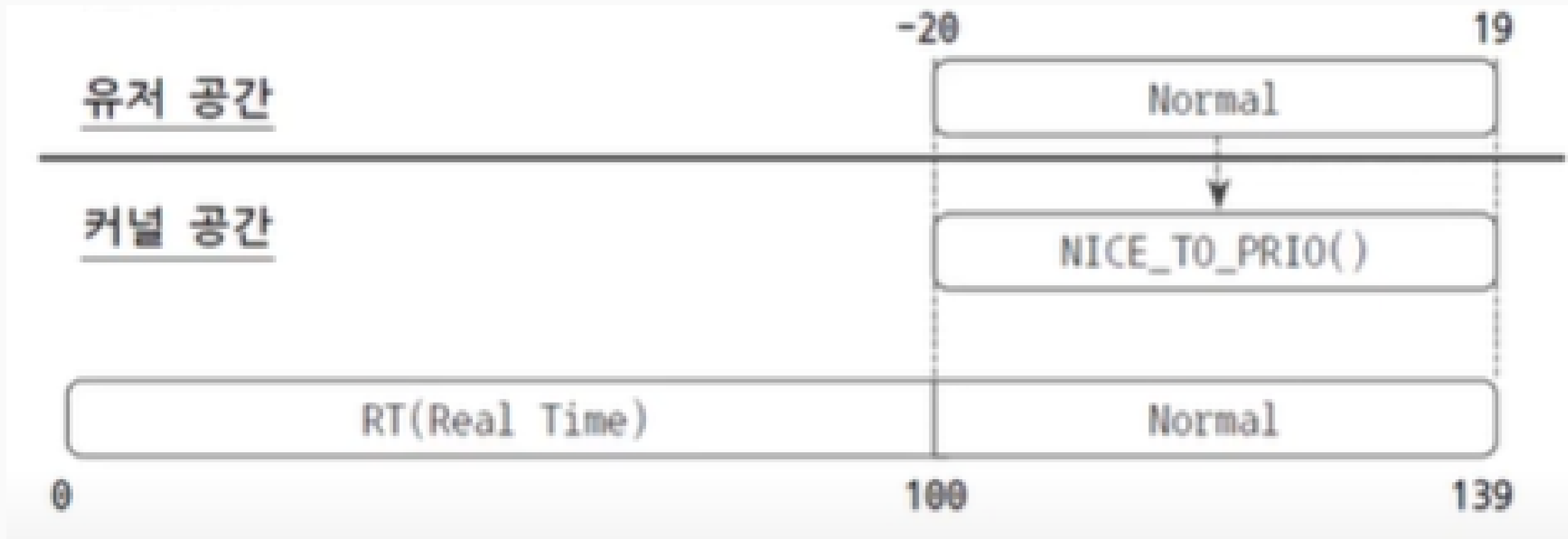
$$vruntime+ = actual\_runtime \times \frac{weight_0}{weight_{task}}$$

기본 우선순위의 가중치  
해당 프로세스의 가중치

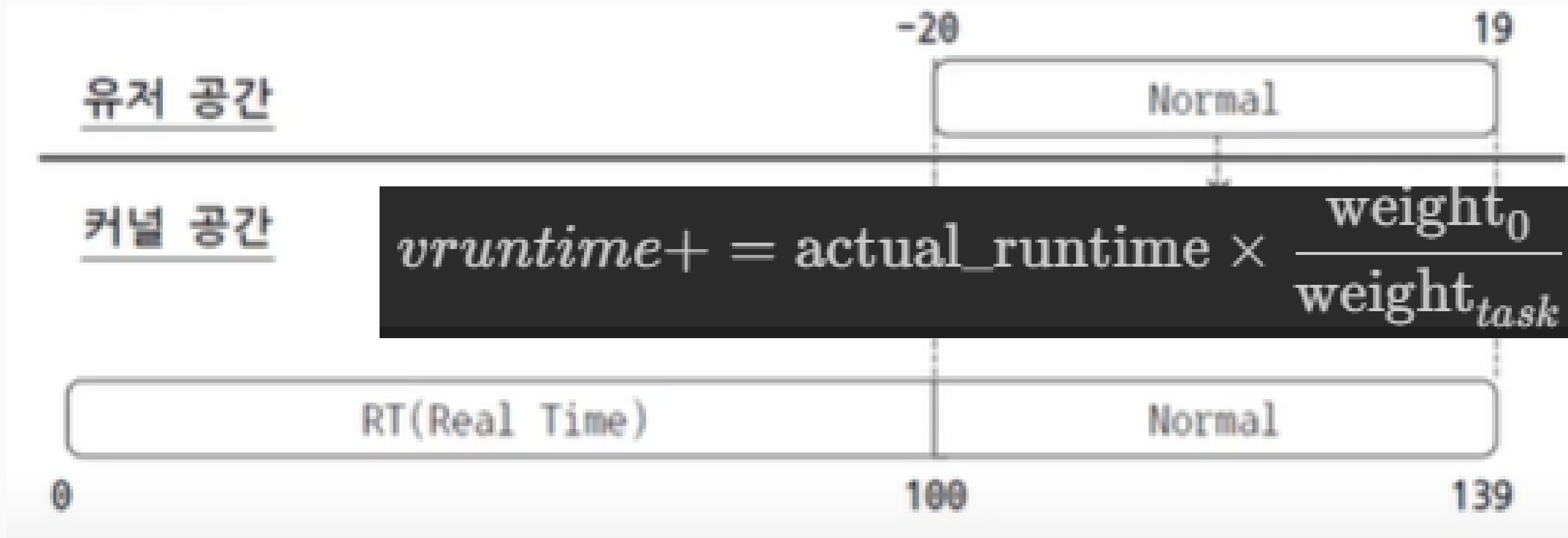
프로세스가 물리적으로  
CPU를 점유한 시간

```
const int sched_prio_to_weight[40] = {  
    /* -20 */      88761,      71755,      56483,  
    /* -15 */      29154,      23254,      18705,  
    /* -10 */      9548,       7620,       6100,  
    /*  -5 */      3121,       2501,       1991,  
    /*   0 */      1024,        820,        655,  
    /*   5 */       335,        272,        215,  
    /*  10 */       110,         87,         70,  
    /*  15 */        36,         29,         23,  
};
```





```
const int sched_prio_to_weight[40] = {  
    /* -20 */      88761,      71755,      56483,      46273,      36291,  
    /* -15 */      29154,      23254,      18705,      14949,      11916,  
    /* -10 */      9548,       7620,       6100,       4904,       3906,  
    /*  -5 */      3121,       2501,       1991,       1586,       1277,  
    /*   0 */      1024,        820,        655,        526,        423,  
    /*   5 */       335,        272,        215,        172,        137,  
    /*  10 */       110,         87,         70,         56,         45,  
    /*  15 */        36,         29,         23,         18,         15,  
};
```

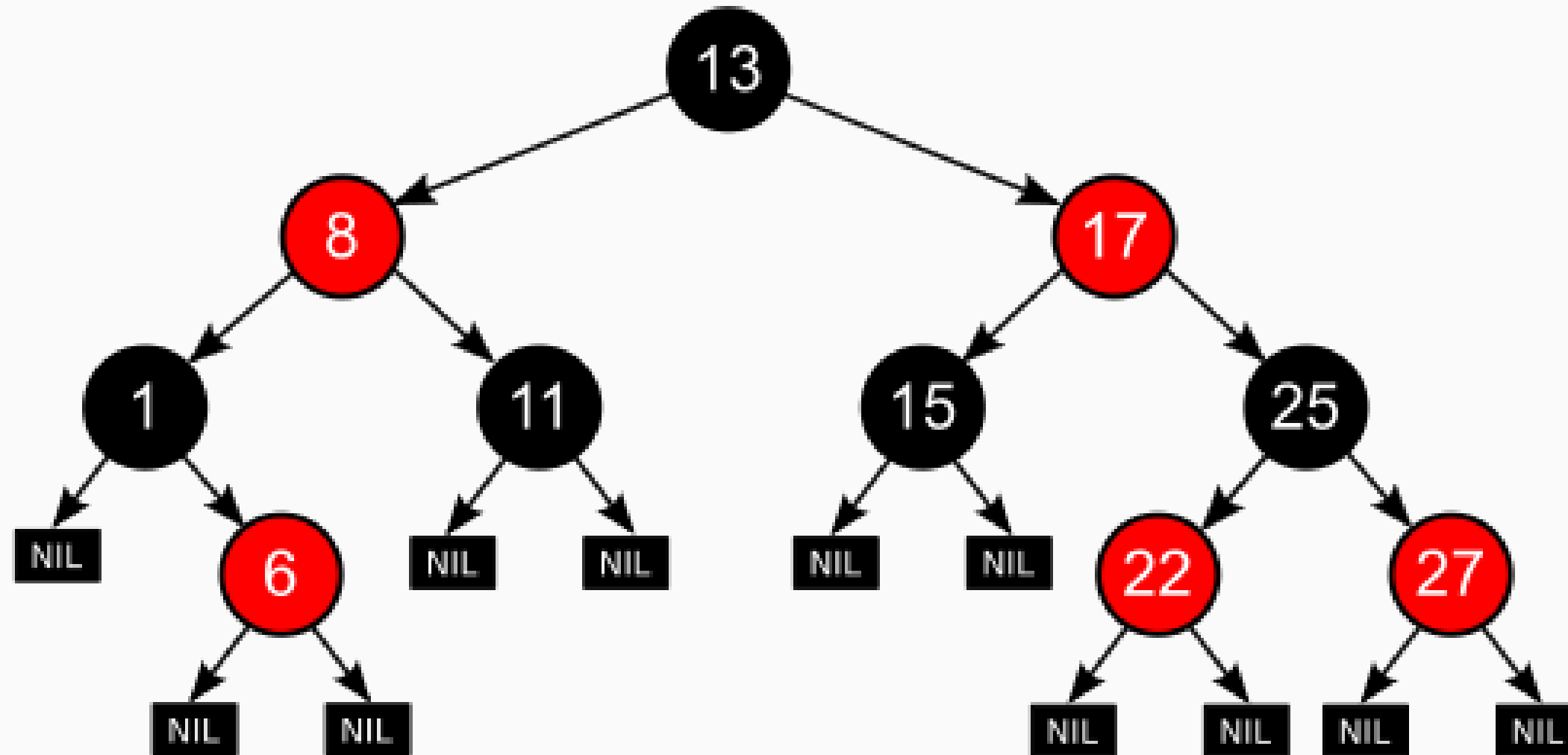


- Nice 0 (기본 우선 순위)
  - 실제 시간 1초가 흐르면 vruntime도 1초 증가
- Nice -10 (우선순위가 높음)
  - 실제 시간 1초가 흐르면 vruntime은 0.1초 정도 증가 → CPU를 더 많이 사용할 수 있음
- Nice +10 (우선순위가 낮음)
  - 실제 시간 1초가 흐르면 vruntime은 10초 정도 증가 → CPU를 거의 사용할 수 없음

# CFS [리눅스 2.6.23 ~ ]

- vruntime이 가장 작은 프로세스를 선택
  - 프로세스가 실행되면 vruntime이 증가하여 다른 대기 중인 프로세스의 vruntime보다 커짐
  - CPU를 반납하면 모든 프로세스의 vruntime은 균형을 맞춤
-

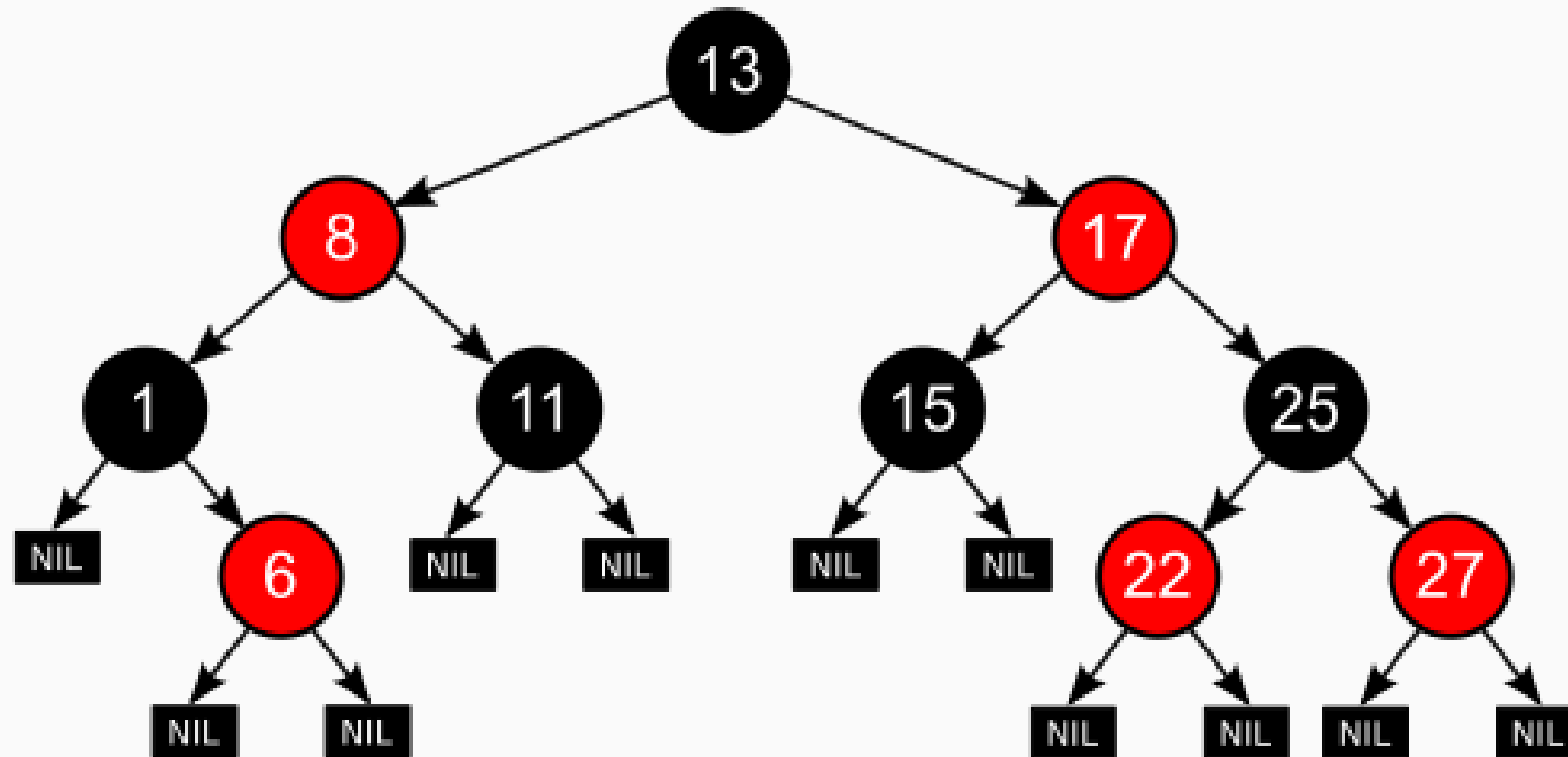
# 레드 블랙 트리



- 자가 균형 이진 탐색 트리
- 삽입, 삭제, 탐색 등 모든 연산에서 최악의 경우에도  $O(\log n)$ 을 보장함

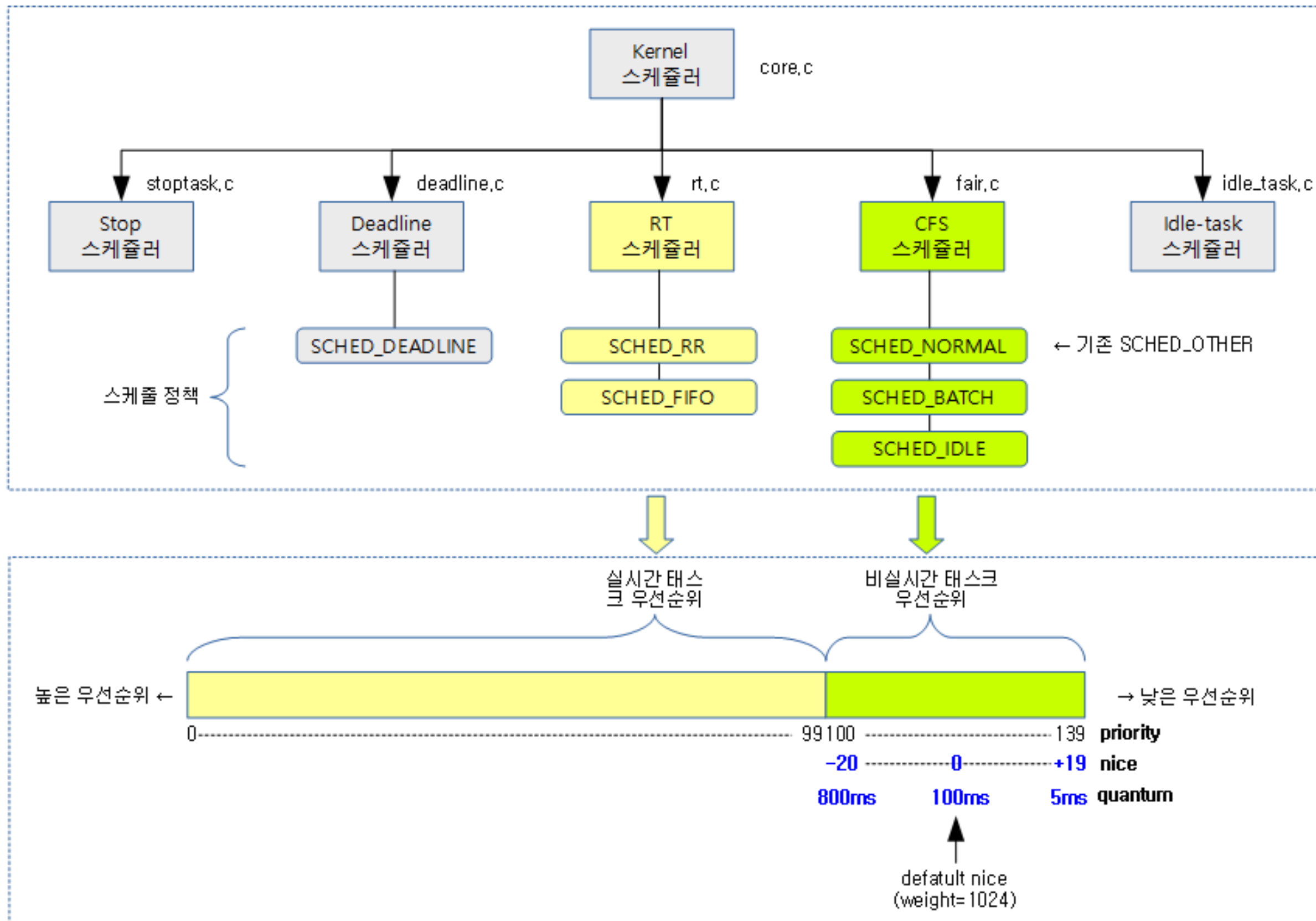


# 레드 블랙 트리



- vruntime이 최소인 트리의 가장 왼쪽 노드를 스케줄러가 선택함
- 선택된 작업이 실행되면 vruntime이 증가함
- 작업이 선점되거나 타임 슬라이스를 소진하면 갱신된 vruntime이 트리에 삽입됨
- 다음에 시작될 프로세스를 선택하는 연산은  $O(1)$

# 스케줄링 정책



- **Deadline 스케줄러**

- 긴급한 시스템 작업을 처리

- **RT 스케줄러**

- 실시간성이 중요한 작업을 처리

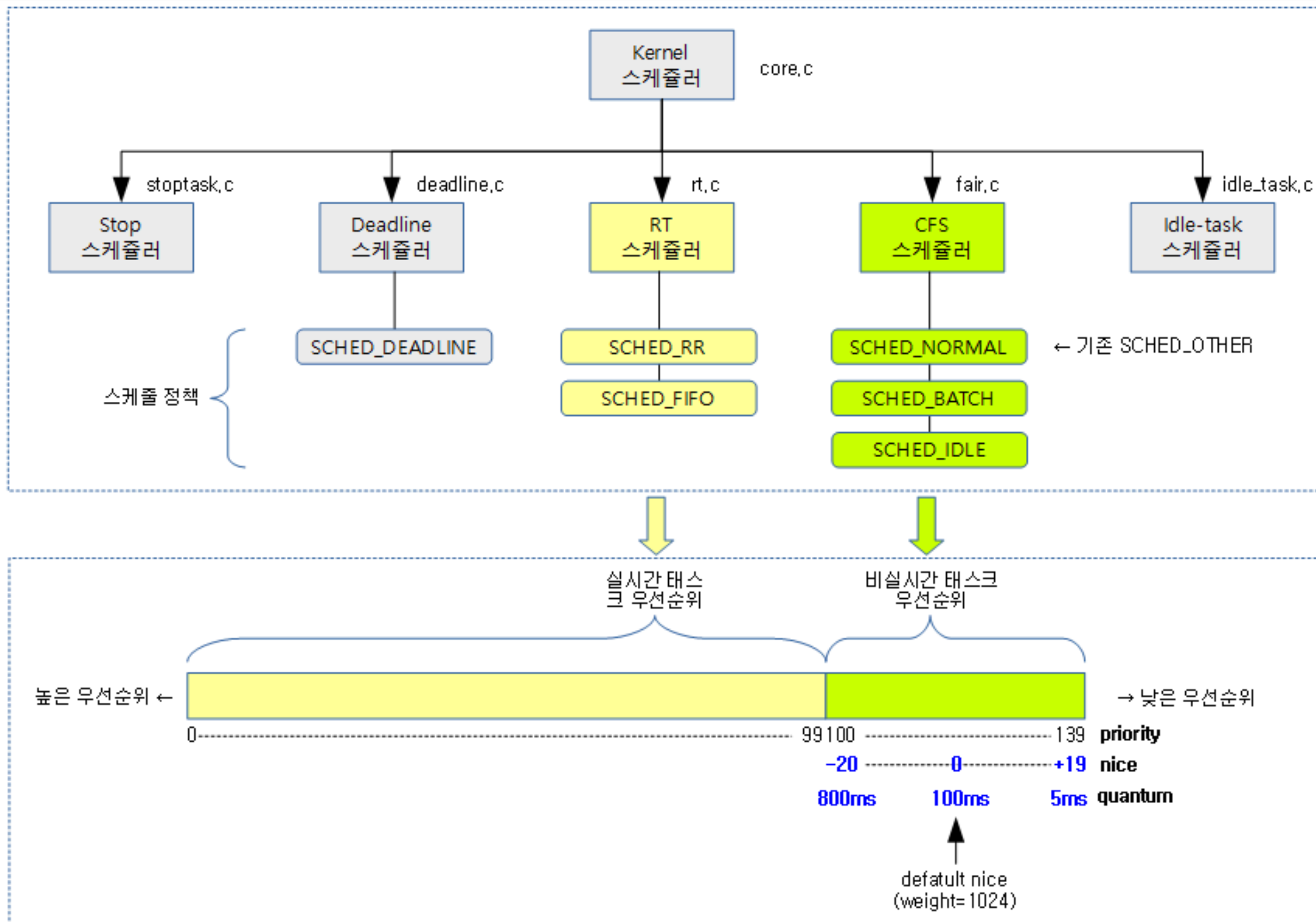
- **SCHED\_FIFO**

- 고정 우선순위의 선입선출 방식
- 타임 슬라이스 개념이 없음

- **SCHED\_RR**

- 같은 우선순위는 타임 슬라이스로 순환하여 사용

# 스케줄링 정책

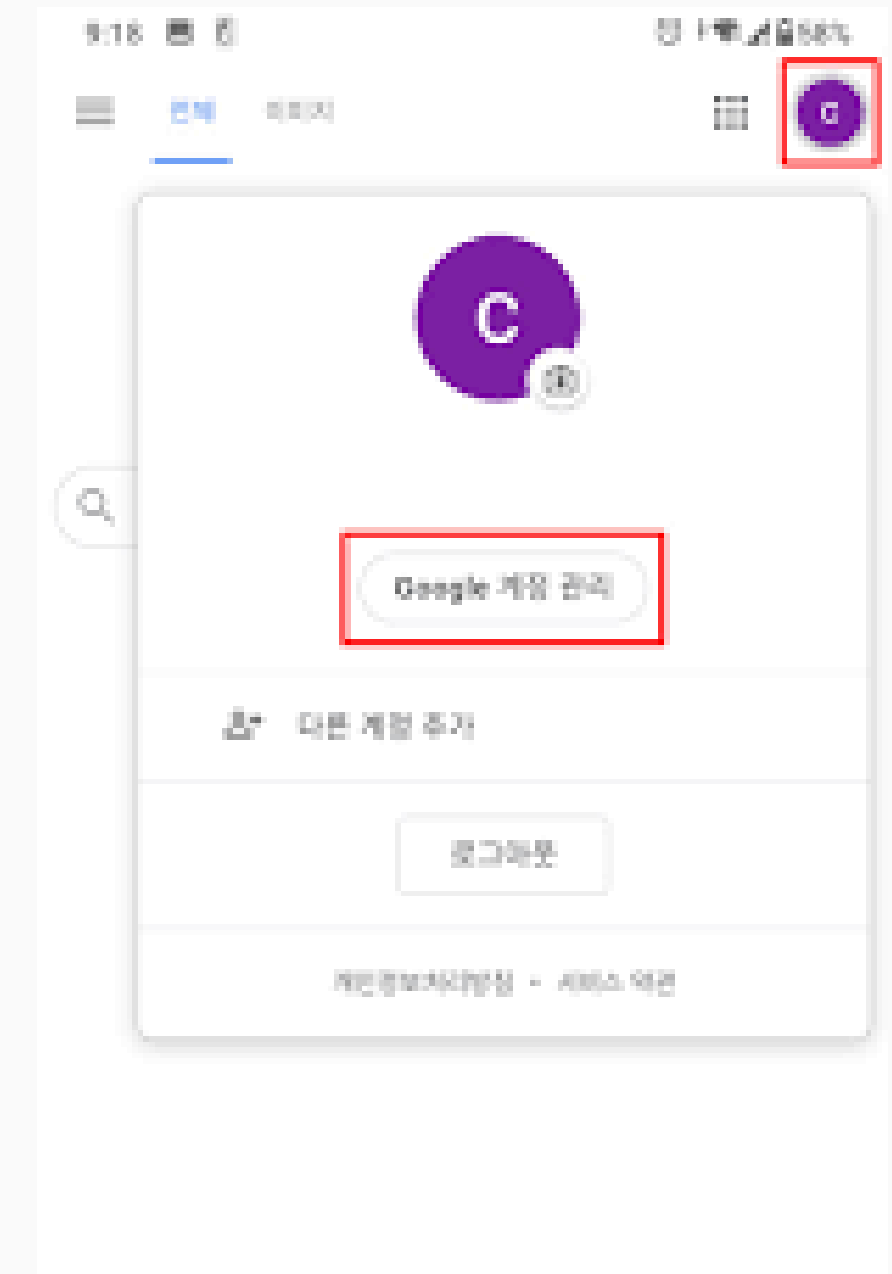
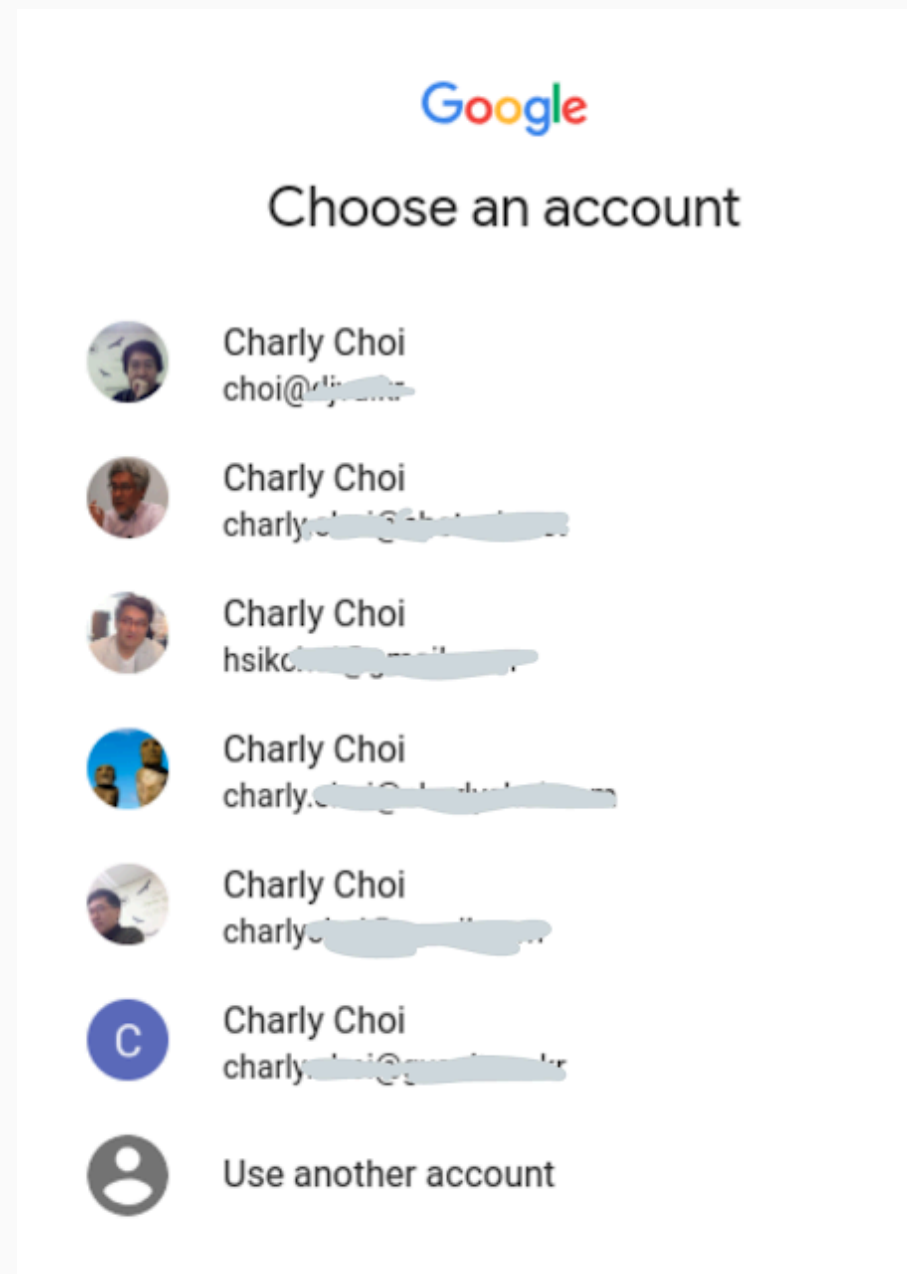


## • CFS 스케줄러

- 일반적인 사용자 프로그램을 처리
- SCHED\_NORMAL
  - 리눅스에서 기본으로 사용
  - vruntime에 기반
- SCHED\_BATCH
  - 배치 작업을 위한 정책
  - 긴 시간동안 CPU를 많이 사용하는 작업에 적합
- SCHED\_IDLE
  - 낮은 우선순위의 작업에 적합
  - 백그라운드 작업

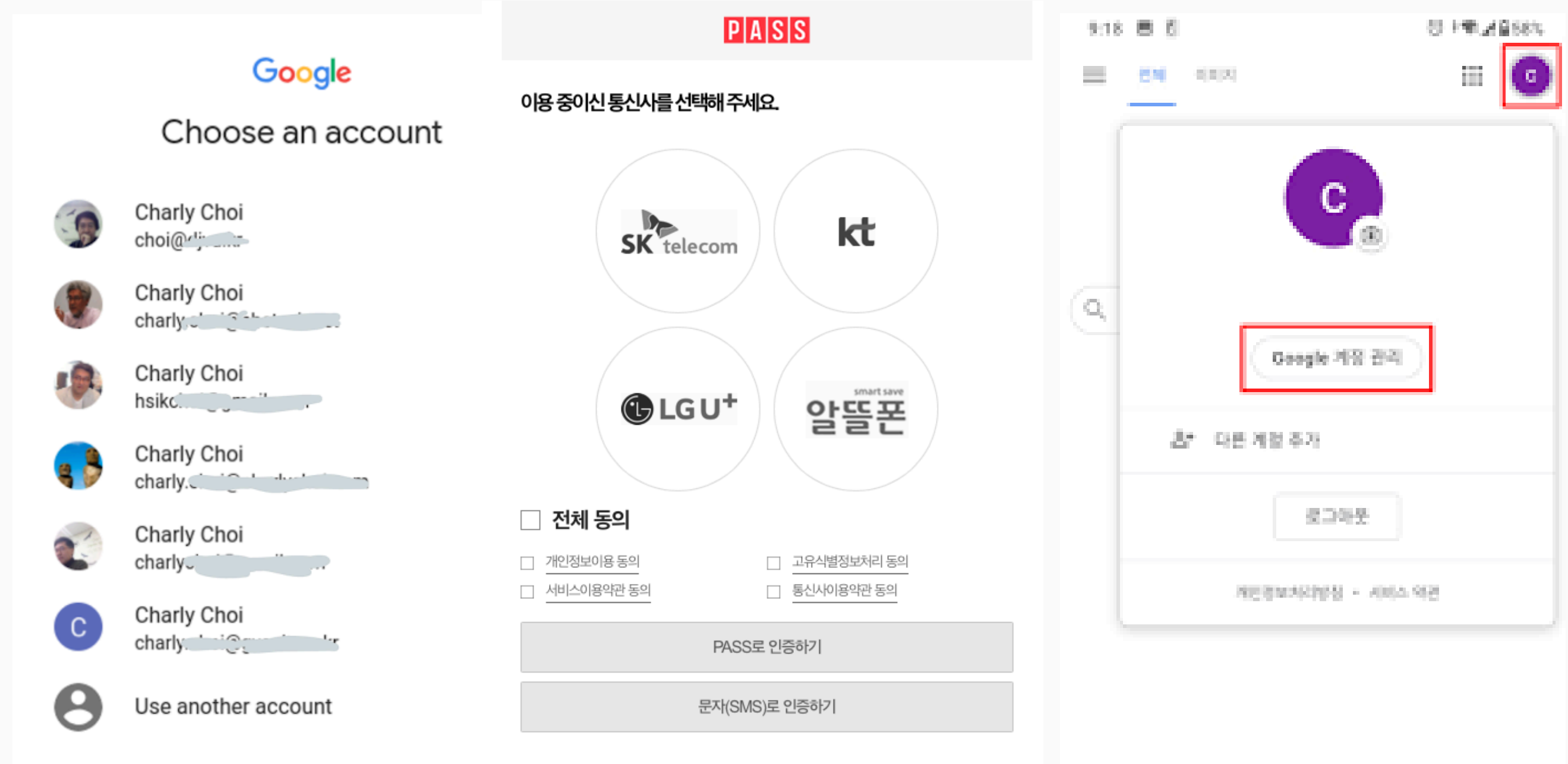


# 그룹 스케줄링



- 사용자 A가 99개의 프로세스를 실행하고 사용자 B가 1개의 프로세스를 실행한다면?
  - 프로세스 단위로 본다면?

# 그룹 스케줄링



- 사용자 A가 99개의 프로세스를 실행하고 사용자 B가 1개의 프로세스를 실행한다면?
  - 그룹 단위로 본다면?

03

EEVDF

# CFS의 한계



- 공정성과 응답성
- 전체 CPU 사용량은 적더라도 즉시 CPU를 사용해야 하는 경우
  - 우선 순위를 높인다는 것은? 더 많은 CPU 시간을 달라는 것

# CFS의 한계

## 코마: 27년 간 혼수상태서 기적 처럼 깨어난 UAE 여성



오랜 의식불명은 대부분 갑작스런 뇌 손상이 원인이다

2019년 4월 24일

1991년 교통사고로 크게 다쳐 27년 동안 혼수상태에 있던 여성이 기적 같이 깨어났다.

- 오랫동안 실행되지 않던 프로세스가 깨어난다면?
  - 실행을 독점할 수 있음
- CFS에서는 현재 최솟값으로 강제 조정하는 방식을 사용
  - Completely **Fair** Scheduler?



# EEVDF [리눅스 6.6 ~]

- Earliest Eligible Virtual Deadline First
  - Lag와 Deadline이라는 개념을 도입하여 CFS 한계를 수학적으로 해결
  - Lag : 프로세스가 이상적인 상황에서 받았어야 할 시간 - 실제 받은 시간
    - $Lag > 0$ 
      - 받아야 할 시간보다 덜 받음 → 실행 자격 있음
    - $Lag < 0$ 
      - 받아야 할 시간보다 더 많이 받음 → 실행 자격 없음
-

# EEVDF [리눅스 6.6 ~ ]

- Earliest Eligible Virtual Deadline First
- Lag와 Deadline이라는 개념을 도입하여 CFS 한계를 수학적으로 해결
- Virtual Deadline : 이 작업을 언제까지 끝내야 공정한지를 계산하여 마감 기한을 부여함

“ 실행 자격이 있는 프로세스들 중에서 가상 마감 기한이 가장 빠른 프로세스 ”

## 자격요건

- B2B 영업경력 2년 이상 혹은 이에 준하는 경력을 가지신 분
- 고객사 관리 경험이 있으신 분
- 문서 작성 능력이 뛰어나신 분

## 우대사항

- 사이버 위협 대응 솔루션 및 유관 서비스 영업 경험이 있으신 분
- 기술에 대한 이해를 바탕으로 고객에게 알기 쉽게 설명할 수 있으신 분
- 국방 또는 공공기관 영업 경력을 보유하신 분
- 지자체 대상 직접영업 경험이 있으신 분
- 지방 협력업체 대상 영업 인프라 구축이 가능하신 분
- 조달 업무 및 영업 경험이 있으신 분

# Latency Nice

- 전체 CPU 할당량은 그대로 유지하면서 스케줄링 빈도를 높여 응답성을 극대화
- Latency Nice를 낮추면?
  - 1회 실행 시간이 짧아짐
  - 할당량이 작으니 **금방 처리해야 할 작업**으로 인식되어 가상 마감 기한이 앞당겨짐
- 
- Latency Nice를 높이면?
  - 실행 시간을 뭉쳐서 제공함
  - 마감 기한이 뒤로 밀려 스케줄러가 **천천히** 처리함

## 자격요건

- B2B 영업경력 2년 이상 혹은 이에 준하는 경력을 가지신 분
- 고객사 관리 경험이 있으신 분
- 문서 작성 능력이 뛰어나신 분

## 우대사항

- 사이버 위협 대응 솔루션 및 유관 서비스 영업 경험이 있으신 분
- 기술에 대한 이해를 바탕으로 고객에게 알기 쉽게 설명할 수 있으신 분
- 국방 또는 공공기관 영업 경력을 보유하신 분
- 지자체 대상 직접영업 경험이 있으신 분
- 지방 협력업체 대상 영업 인프라 구축이 가능하신 분
- 조달 업무 및 영업 경험이 있으신 분



2025.11.14

# 감사합니다

operating system